

Phase 6 — Containerize & Deploy on GCP (single VM → GKE autoscaling)

Goal of this phase: package the server in a container and run it on Google Cloud — first on a single GPU VM (simplest), then on GKE with a Horizontal Pod Autoscaler so it scales under load. Every config is prepared and YAML-validated on CPU; this doc is the runbook for the paid session.

Status: Dockerfiles + GKE manifests written and validated on CPU (no spend). Run during a focused, budget-aware cloud session. **GPUs bill by the hour — follow the teardown checklist every time.**

1. Plain-language concepts (read once)

- **Container (Docker):** a sealed box with the code + every dependency, so it runs identically on my laptop and in the cloud. Built from a **Dockerfile** (a recipe). The image is pushed to a **registry** (Artifact Registry on GCP) and pulled by the machine that runs it.
 - **Compute Engine VM:** one rented Linux machine. Simplest place to run the container. I attach one GPU and use a **spot/preemptible** VM (much cheaper; can be reclaimed — fine for benchmarks).
 - **Kubernetes (GKE):** an orchestrator that runs containers across machines and keeps the desired number alive. It gives me **autoscaling**: add pods (and GPU nodes) under load, remove them when idle.
 - **Pod / Deployment / Service / HPA:** a *pod* runs my container; a *Deployment* keeps N pods running; a *Service* gives them one stable external IP (LoadBalancer); an *HPA* changes N based on a metric.
-

2. What I built (validated on CPU)

- **deploy/Dockerfile** — GPU serving image (CUDA runtime base; on Linux the default torch wheel is the CUDA build, so it's GPU-ready). Layer order puts deps before code so rebuilds are fast.
 - **deploy/Dockerfile.cpu + deploy/docker-compose.yml** — one-command local run on a laptop.
 - **requirements-serve.txt** — slim runtime deps (no test/plot/docs bloat in the image).
 - **.dockerignore** — keeps weights, venv, results, docs out of the build context.
 - **deploy/gke/{deployment,service,hpa}.yaml** — GPU pod (1 GPU/pod, /health probes), external LoadBalancer, and an HPA (1→4 pods). All three YAML-validated.
-

3. Dockerfile, layer by layer

```
FROM nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04 # CUDA/cuDNN libs torch needs on a GP
ENV PYTHONUNBUFFERED=1 HF_HOME=/models ... # unbuffered logs; cache weights on a
RUN apt-get ... python3 python3-pip # the CUDA image has no Python
COPY requirements-serve.txt . # copy deps file alone first...
RUN pip3 install -r requirements-serve.txt # ...so this slow layer caches until
COPY engine/ engine/ ; COPY server/ server/ # app code copied after deps (changes
EXPOSE 8000
CMD ["uvicorn","server.app:app","--host","0.0.0.0","--port","8000"] # 0.0.0.0 = reachabl
```

The key idea is **layer caching**: dependencies (slow) are a separate layer from code (fast-changing), so editing code rebuilds in seconds.

4. RUNBOOK A — single GPU VM (do this first)

One-time setup:

```
gcloud auth login # authenticate
gcloud config set project PROJECT_ID # pick your project
gcloud services enable compute.googleapis.com artifactregistry.googleapis.com
gcloud billing budgets create ... --display-name "llm-200" \
--budget-amount 200USD # ALERT at $200 (set this first)
```

Build & push the image (Artifact Registry):

```
gcloud artifacts repositories create inference --repository-format=docker --location=REGI
gcloud auth configure-docker REGION-docker.pkg.dev
docker build -f deploy/Dockerfile -t REGION-docker.pkg.dev/PROJECT_ID/inference/inference
docker push REGION-docker.pkg.dev/PROJECT_ID/inference/inference-engine:latest
```

Create a cheap spot GPU VM (T4/L4 is plenty for a 0.5B model):

```
gcloud compute instances create infer-vm \
  --zone=ZONE \
  --machine-type=g2-standard-4 \           # L4 family; or n1-standard-4 + --acc
  --accelerator=type=nvidia-l4,count=1 \   # one GPU
  --provisioning-model=SPOT \              # cheap, preemptible – fine for benc
  --maintenance-policy=TERMINATE \
  --image-family=common-gpu-debian-11 \    # comes with NVIDIA drivers preinstal
  --image-project=deeplearning-platform-release \
  --boot-disk-size=100GB
```

Each flag: `--accelerator` attaches the GPU; `--provisioning-model=SPOT` is the cost saver; `--maintenance-policy=TERMINATE` is required for GPUs; the deeplearning image ships GPU drivers so I don't install CUDA by hand.

Run the container on the VM:

```
gcloud compute ssh infer-vm --zone=ZONE      # shell into the box
nvidia-smi                                   # confirm the GPU is visible
docker run -d --gpus all -p 8000:8000 \      # --gpus all exposes the GPU to the c
  REGION-docker.pkg.dev/PROJECT_ID/inference/inference-engine:latest
curl localhost:8000/health                   # confirm it's serving
```

Hit it from your laptop (open the port, then call the external IP):

```
gcloud compute firewall-rules create allow-8000 --allow tcp:8000
gcloud compute instances describe infer-vm --zone=ZONE \
  --format='get(networkInterfaces[0].accessConfigs[0].natIP)' # external IP
curl http://EXTERNAL_IP:8000/health
```

 **Confirm the VM path works before touching GKE.** Then run the Phase 5 GPU benchmarks against it.

5. RUNBOOK B — GKE with autoscaling

Create a cluster with a GPU node pool that can scale to zero-ish:

```

gcloud container clusters create infer-cluster --zone=ZONE --num-nodes=1 # small default
gcloud container node-pools create gpu-pool \
  --cluster=infer-cluster --zone=ZONE \
  --machine-type=g2-standard-4 --accelerator=type=nvidia-l4,count=1 \
  --enable-autoscaling --min-nodes=0 --max-nodes=3 \ # node autoscaler adds GPUs under
  --spot # spot GPU nodes = cheaper
gcloud container clusters get-credentials infer-cluster --zone=ZONE # point kubectl at
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/container-engine-a

```

Deploy (set the image first):

```

sed -i 's#REGION-docker.pkg.dev/PROJECT_ID#REGION-docker.pkg.dev/YOURPROJECT#' deploy/gke
kubectl apply -f deploy/gke/deployment.yaml
kubectl apply -f deploy/gke/service.yaml
kubectl apply -f deploy/gke/hpa.yaml
kubectl get pods -w # wait for Running + Ready
kubectl get service inference-engine # grab EXTERNAL-IP

```

Two layers of autoscaling to understand: 1. **HPA** adds *pods* when avg CPU > 60% (deploy/gke/hpa.yaml). 2. **Cluster autoscaler** adds *GPU nodes* when a new pod can't be scheduled (each pod needs 1 GPU).

6. Drive load and watch it autoscale (great for the README)

```

# point the existing harness at the GKE external IP
python benchmark/runner.py --url http://EXTERNAL_IP --concurrency 1,4,16,64 \
  --requests 128 --max-tokens 128 --engines batched

# in another terminal, watch scaling happen
kubectl get hpa inference-engine -w # REPLICAS climbs as load rises
kubectl get pods -w # new pods appear (then new nodes if needed)

```

Capture `kubectl get hpa` before/after for the README — "replicas went 1 → 4 under load" is the demo.

7. 🚫 TEARDOWN CHECKLIST (run every single time — GPUs bill hourly)

```
# GKE
kubectl delete -f deploy/gke/                # remove workloads + LoadBalancer
gcloud container clusters delete infer-cluster --zone=ZONE  # deletes node pools too

# Single VM
gcloud compute instances delete infer-vm --zone=ZONE
gcloud compute firewall-rules delete allow-8000

# Verify NOTHING is still running / billing
gcloud compute instances list                # expect: empty
gcloud container clusters list              # expect: empty
```

- ☐ Cluster deleted - ☐ Node pools gone (deleted with cluster) - ☐ VM deleted
- ☐ LoadBalancer deleted (it's a billed resource) - ☐ Billing console shows nothing running

8. Interview Q&A

Q: Why containerize at all? A: Reproducibility and portability — the image bundles code + CUDA-compatible deps so it runs the same on my laptop, a VM, and GKE. No "works on my machine," and deploys are just "pull this image."

Q: Why VM first, then GKE? A: A single VM is the simplest way to prove the container runs on a GPU and serves traffic. GKE adds orchestration and autoscaling but also complexity — I de-risk the container on a VM before layering on Kubernetes.

Q: What are the two autoscaling layers in GKE? A: The HPA scales *pods* based on a metric (CPU% here); the cluster autoscaler scales *nodes* when pods can't be scheduled. For GPU serving they work together — more pods need more GPU nodes.

Q: Why scale on CPU and not GPU utilization? A: CPU% needs no extra plumbing and demonstrates autoscaling. In production I'd scale on a serving signal — request queue depth or tokens/sec — exposed as a custom metric via the Prometheus Adapter or Cloud Monitoring, because GPU work doesn't always track CPU.

Q: How do you control cost? A: Spot/preemptible instances, a small GPU sufficient for the model, a budget alert at \$200, max-replica caps so autoscaling can't run away, and a teardown checklist I run after every session. GPUs idle at ~\$3–4/hr for an A100, so "never leave it running" is the rule.

Q: What does the readiness probe protect against? A: The model takes ~30s+ to load. The readiness probe on `/health` keeps the pod out of the Service's rotation until it can actually serve,

so no requests hit a cold, not-yet-loaded pod.

9. One-line recall

"I containerized the server (CUDA base, cached dependency layers), ran it on a spot GPU VM, then on GKE with a Deployment, LoadBalancer Service, and an HPA — and drove my own load harness at it to watch pods autoscale 1→4. I scale on CPU for simplicity but know the production move is a queue-depth/tokens custom metric, and I have a teardown checklist so a forgotten GPU never bills."